

YOUR CODING AGENT SAYS

01 / 05

"The prompt is broken."

Maybe. A failure can start in **nine** places.
Editing the prompt fixes **one**.

EVAL HARNESS

IMPROVEMENT LOOP

CROSS-VENDOR JUDGES

MEMORY

9

FAILURE ORIGINS

18

MCP TOOLS

66

STARTER CARDS

≥2

VENDORS PER SCORE

ENDPOINT

01 Su	02 Cg	03 Ra
04 Jv	05 Fm	06 If
07 Pp	08 Sa	09 Ce

Titration finds which one, with judges from
other vendors.

open source · Apache-2.0
self-hosted · any MCP client

THE NINE FAILURE ORIGINS

The periodic table of *agent failure*.

One failure, one origin, one place to fix it.

01 MODEL

Su

system-under-test
the model really got it wrong

fix → the prompt

The only one that means “edit the prompt.”

Misread another as **01** and you fix the wrong thing.

02 TEST SET

Cg

corpus-gap
the tests never exercise the case
fix → the corpus

03 GRADING

Ra

rubric-ambiguity
too vague to judge consistently
fix → the rubric

04 GRADING

Jv

judge-variance
judges disagree below the noise floor
fix → the judges

05 PIPELINE

Fm

formatter
code after the model broke good output
fix → the formatter

09 RUNTIME

Ce

cost-exhausted
the loop ran out of budget
fix → the budget

Eight of nine are not a prompt problem.

Split evidence **never defaults to 01.**

07 TEST SET

Pp

pre-existing-pattern
already failing in the baseline
fix → not this change

06 GRADING

If

instrument-failure
the measurement itself is wrong
fix → the harness

FAMILIES

- model
- test set
- grading
- pipeline
- runtime

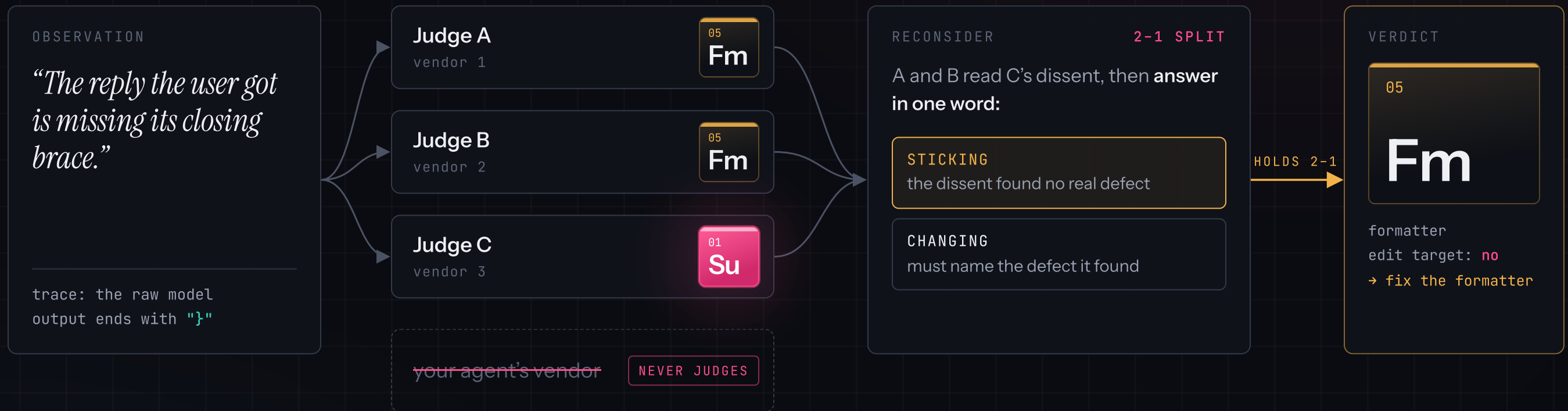
08 PIPELINE

Sa

state-artifact
an earlier turn corrupted this one
fix → upstream

— HOW IT DECIDES

Three judges, three vendors. A split gets *argued*, not averaged.



Judges diagnose, never prescribe.

The fix is yours to choose.

Confidence is not evidence.

Changing an answer means naming the defect.

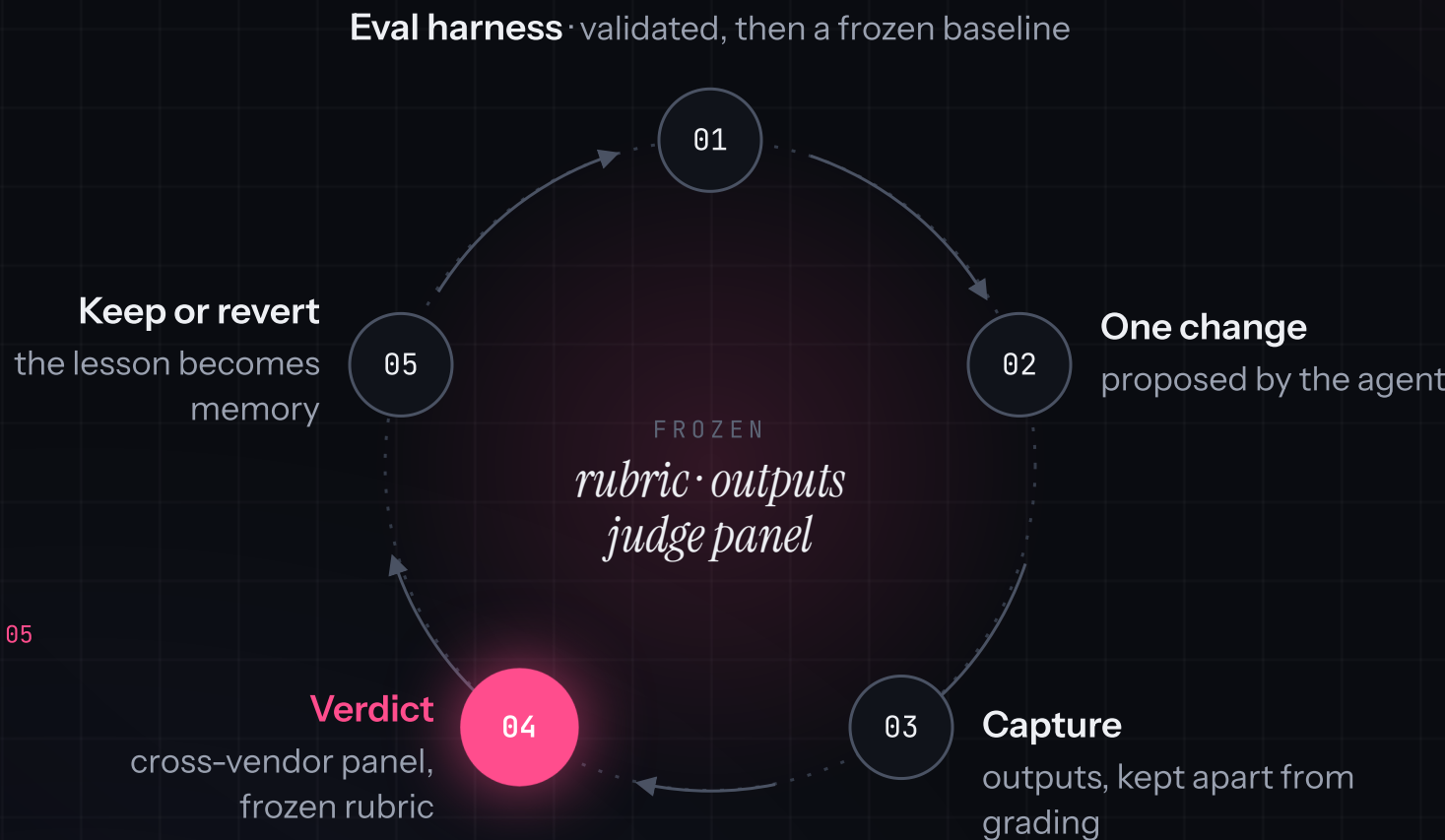
Nothing is thrown away.

Both rounds stay in the audit trail.

THE IMPROVEMENT LOOP

One change at a time, against a baseline that *doesn't move*.

"When a measure becomes a target, it ceases to be a good measure." GOODHART'S LAW · MARILYN STRATHERN



RULES OF EVIDENCE

- i.* **The model never grades its own work.**
Your agent's vendor never sits on the panel.
- ii.* **Agreement, or no score.**
At least two vendor families must answer, or there is no score.
- iii.* **Memory that compounds.**
Findings become cards, searched with 66 starter methods.

90% → 0%

REAL RUN · EXAMPLES/TICKET-TRIAGE
Billing tickets wrongly marked urgent, fixed in one measured change. Noise floor ± 1.1 pts, 90 of 90 votes.

 GET STARTED

On your machine in *about five minutes.*

```
~/titration
$ git clone https://github.com/kaithoughtarchitect/titration.git
$ cd titration
$ docker compose up -d          # Postgres + pgvector
$ npm install
$ cp .env.example .env          # add OPENROUTER_API_KEY
$ npm run setup

# connect your agent, e.g. Claude Code
$ claude mcp add titration -- npx tsx "$PWD/server/server.ts"
```

- New here? Start with `examples/ticket-triage`: two planted bugs, one measured fix.

“The first principle is that you must not fool yourself and you are the easiest person to fool.”

RICHARD FEYNMAN

YOU NEED

- **Node.js 22.11+** and **Docker**
- An **OpenRouter** key. The **claude · codex · grok** CLIs can fill up to two judge seats.

HELP BUILD IT

- Add a learning to the starter pack
- Verify the **grok** judge door
- Run it on a real agent change, open an issue

STAR · FORK · CONTRIBUTE

github.com/kaithoughtarchitect/titration